Tytuł: **RoboZap Game**

Autorzy: **Łukasz Perczyński (ŁP), Tymon Ryś (TR)**

Ostatnia modyfikacja: 01.09.2024

Spis treści

1. Repozytorium git 1
2. Wstęp 1
3. Specyfikacja 1
   3.1. Opis ogólny algorytmu 1
   3.2. Tabela zdarzeń 2
4. Architektura 2
   4.1. Moduł: top 2
      4.1.1. Schemat blokowy 2
      4.1.2. Porty 3
         a) mou – mouse_ctl, input 3
         b) vga – vga_ctl, output 3
      4.1.3. Interfejsy 3
         a) m2c – mouse_ctl to core 3
   4.2. Rozprowadzenie sygnału zegara 3
5. Implementacja 4
   5.1. Lista zignorowanych ostrzeżeń Vivado. 4
   5.2. Wykorzystanie zasobów 4
   5.3. Marginesy czasowe 4
6. Film. 4

1. Repozytorium git

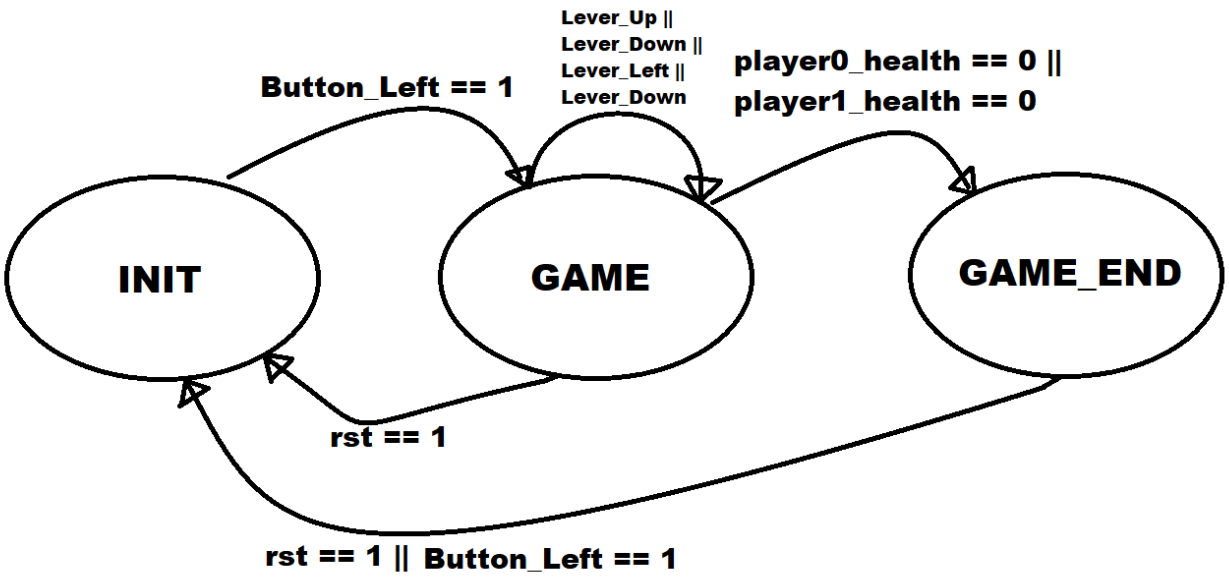
Adres repozytorium GITa:

https://github.com/TrippyVoid29/RoboZapGame

2. Wstęp

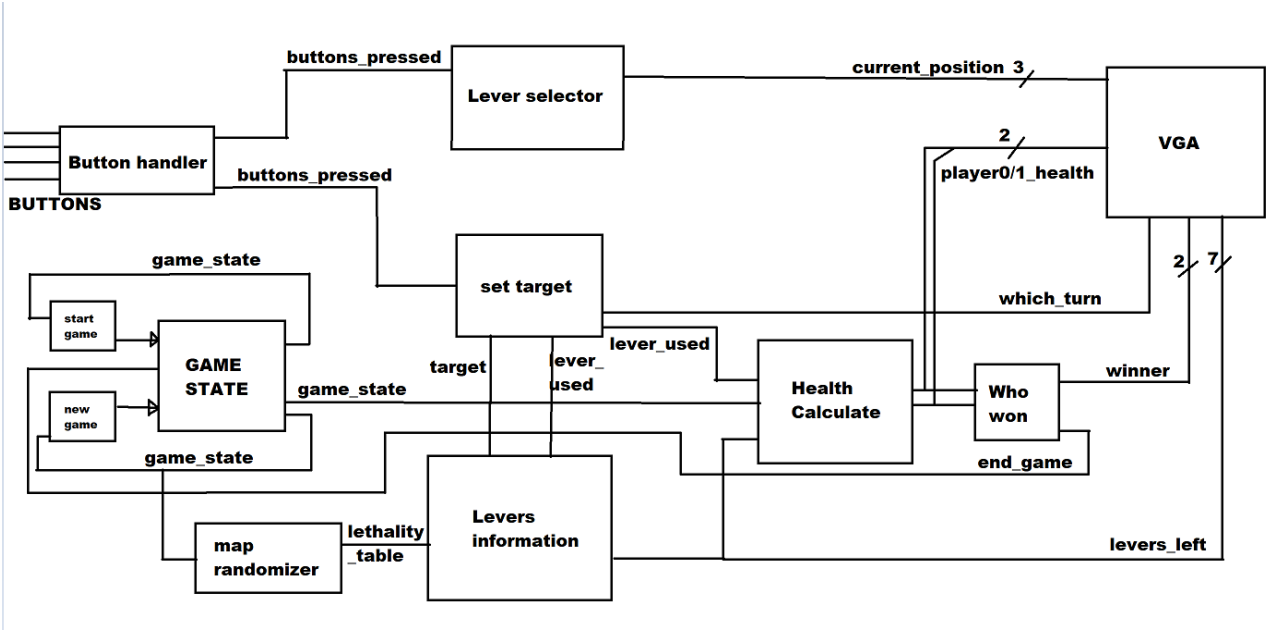
Nasz projekt to prosta gra napisana w języku SystemVerilog wzorująca się na grze typu "rosyjska ruletka", natomiast zamiast rewolwera na ekranie znajduje się 8 dźwigni, które można przeciągnąć w stronę swoją (ButtonDown), lub przeciwnika (ButtonUp). Zmianę dźwigni dokonuje się za pomocą przycisków ButtonLeft i ButtonRight. Reset synchroniczny działa na przycisku ButtonCenter. Celem gry jest wyeliminowanie przeciwnika. Każda dźwignia może zadać obrażenia osobie na którą została przeciągnięta, ale także może nic nie zrobić. W wypadku przeciągnięcia dźwigni na swoją stronę pozyskujemy dodatkową turę, ale wiąże się to z ryzykiem. Pozycje "śmiertelnych" dźwigni są losowane przy każdym rozpoczęciu rozgrywki.

.



3. Specyfikacja

3.1. Opis ogólny algorytmu



Uproszczony schemat blokowy działania implementowanego algorytmu.

3.2. Tabela zdarzeń

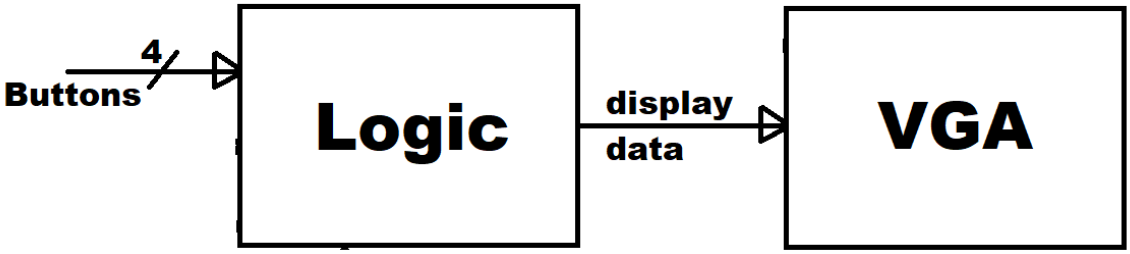
Zdarzenie	Kategoria	Reakcja systemu
ButtonL	INIT	Uruchomienie gry
ButtonL	GAME	Zmiana wyboru dźwigni w lewo
ButtonR	GAME	Zmiana wyboru dźwigni w prawo
ButtonU	GAME	Przeciągnięcie dźwigni na przeciwnika
ButtonD	GAME	Przeciągnięcie dźwigni na siebie
Przeciągnięcie dźwigni	GAME	Zadanie obrażeń/brak obrażeń/wyłączenie możliwości korzystania z danej dźwigni
Dźwignia na siebie	GAME	Pozostanie w swojej turze
Dźwignia na przeciwnika	GAME	Przejście do tury przeciwnika
Wyzerowanie życia któregoś z graczy	GAME	Zakończenie gry, przejście do stanu GAME_END, wyświetlenie informacji kto wygrał
ButtonL	GAME_END	Rozpoczęcie nowej rozgrywki

4. Architektura

4.1. Moduł: top

Osoba odpowiedzialna: ŁP i TR

4.1.1. Schemat blokowy



4.1.2. Porty

) Logic – logic_ctl, input

nazwa portu	opis
ButtonC	reset
ButtonD/U/L/R	Sterowanie leverami

) vga – vga_ctl, output

nazwa portu	opis
vga_vs	sygnał synchronizacji pionowej VGA
vga_hs	sygnał synchronizacji poziomej VGA
{r,g,b}	Interfejs wyświetlacza

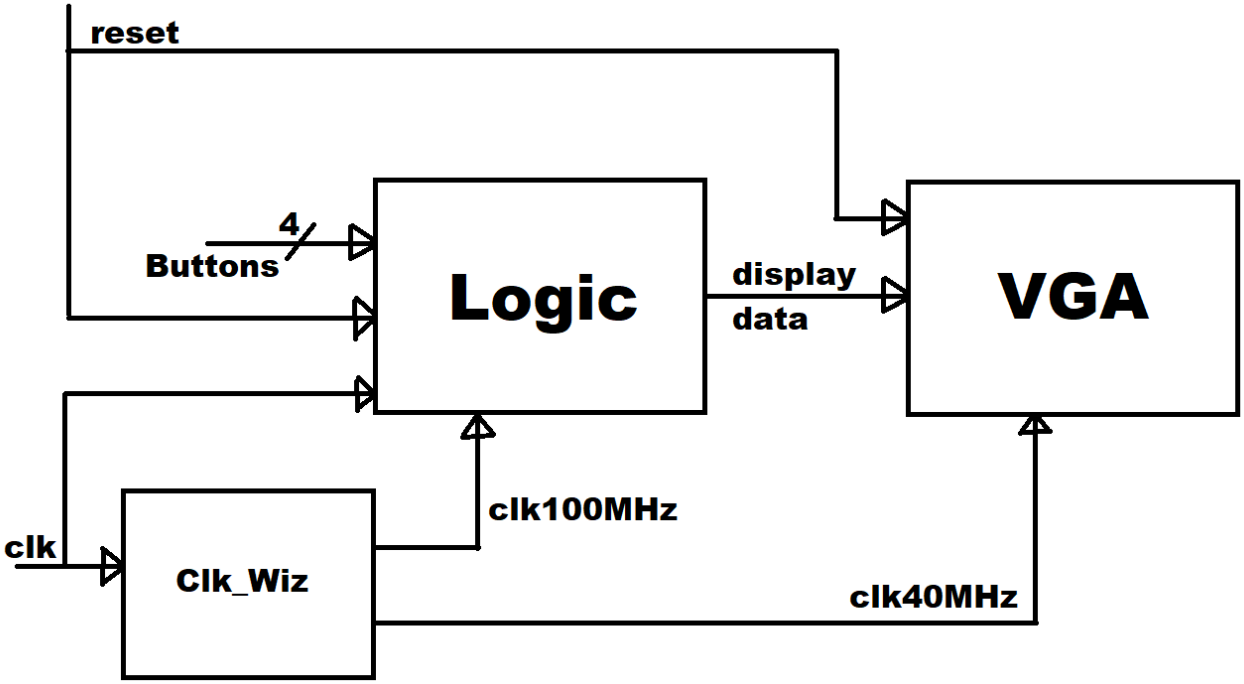
4.1.3. Interfejsy

) Vga_if – vga_ctl

nazwa sygnału	opis
Vga_if	Przesyłanie sygnałów potrzebnych do wyświetlania obrazu

4.2. Rozprowadzenie sygnału zegara

Osoba odpowiedzialna: TR i ŁP



Do wygenerowania sygnału zegara wykorzystaliśmy IP Vivado, a dokładniej Clk_wizard. Generuje nam on zegary o zadanych częstotliwościach: 100 MHz i 40 MHz. VGA jest sterowane zegarem 40 MHz dopasowanym do częstotliwości wyświetlacza, a moduł logiki pracuje na szybszym 100 MHz zegarze.

5. Implementacja

5.1. Lista zignorowanych ostrzeżeń Vivado.

Identyfikator ostrzeżenia	Liczba wystąpień	Uzasadnienie
Synth 8-3838	3	Wynika z wykorzystania modułu napisanego na laboratorium. (wykorzystanie interfejsu vga_if)
Synth 8-87	13	Wynika z wykorzystania modułu napisanego na laboratorium. Nic nie zmienia w funkcjonalności kodu (może wprowadzić niezauważalne opóźnienia)
Synth 8-6014	1	Wynika z rozróżnienia zmiennej wewnętrznej do obsługi maszyny stanów, a sygnału wyjściowego.

Synth 8-7129	72	Sygnał position został dodany do vga_if w celu uniknięcia Negative Slack (wykorzystaliśmy te połączenia jako bufor opóźniający sygnał)
Netlist 29-345	4	Wynika z wykorzystania modułu clk_wiz.
Synth 8-327	19	Zapewnienie ciągłości sygnału tuż po resecie, dla bezpieczeństwa powtórzenie przypisania wartości w innych stanach FSM.
Synth 8-7080	1	Warning nie mający żadnego poważnego wpływu na układ. Można go potraktować jak INFO.
Synth 8-3332	1	Nie wykorzystaliśmy pojedynczego bitu podczas ustalania sygnału {r,g,b}.
DRC PDRC-153	10	Z powodu wielkości naszego projektu nie ma większego wpływu na wydajność.

5.2. Wykorzystanie zasobów

Utilization Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	524	20800	2.52
LUTRAM	4	9600	0.04
FF	256	41600	0.62
IO	20	106	18.87
BUFG	3	32	9.38
MMCM	1	5	20.00

5.3. Marginesy czasowe

Timing **Setup** | Hold

Worst Negative Slack (WNS): 2.062 ns

Timing Setup | **Hold**

Worst Hold Slack (WHS): 0.106 ns

6. Film.

Link do ściągnięcia filmu:

https://drive.google.com/file/d/18yD7MS4PN_eWpNdfHkNRDjpp784FJEIy/view?usp=sharing